Ubuntu 24.04.3 LTS

Mar 3 05:32

```c
static const char* intro_text(void) {
    return
        "It also reinforced the importance of validating each step (like checking open handles)
        "Overall, it gave me a clearer picture of how real file systems manage data and enforce
}

// prints the menu
static void print_menu(int fd_state) {
    printf("\n============================\n");
    printf(" libFC Test App (Diego)\n");
    printf("============================\n");
    printf("File: %s\n", FILENAME);
```

```
./diego testapp
diego@codebind:~$ cd '/home/diego/cs470/Lab5'
diego@codebind:~/cs470/Lab5$ make
./diego_testapp
gcc -Wall -Wextra -std=c11 -g -c Diego_testFC.c -o Diego_testFC.o
gcc -Wall -Wextra -std=c11 -g -c Diego_libFC.c -o Diego_libFC.o
gcc -Wall -Wextra -std=c11 -g -o diego_testapp Diego_testFC.o Diego_libFC.o
Welcome. This program tests the file control functions.

============================
 libFC Test App (Diego)
============================
File: DiegoG_Trevino_Introduction.txt
Open handle state: CLOSED

1) fileCreate  (create new file)
2) fileOpen    (open existing file)
3) fileWrite   (write introduction)
4) fileRead    (read + print file)
5) fileClose   (close file)
6) fileDelete  (delete file)
0) Exit

Enter choice:
```

Ubuntu 24.04.3 LTS

Mar 3 05:33

```c
static const char* intro_text(void) {
    return
        "It also reinforced the importance of validating each step (like checking open handles)
        "Overall, it gave me a clearer picture of how real file systems manage data and enforce
}

// prints the menu
static void print_menu(int fd_state) {
    printf("\n============================\n");
    printf(" libFC Test App (Diego)\n");
    printf("============================\n");
    printf("File: %s\n", FILENAME);
```

```
diego@codebind:~/cs470/Lab5$ make
./diego testapp
 libFC Test App (Diego)
============================
File: DiegoG_Trevino_Introduction.txt
Open handle state: CLOSED

1) fileCreate  (create new file)
2) fileOpen    (open existing file)
3) fileWrite   (write introduction)
4) fileRead    (read + print file)
5) fileClose   (close file)
6) fileDelete  (delete file)
0) Exit

Enter choice: 1

Creating file...
File created successfully.

Press ENTER to continue...

============================
 libFC Test App (Diego)
```

Ubuntu 24.04.3 LTS

Mar 3 05:37

File Edit Selection View Go ... diego

EXPLORER

DIEGO
- .cache
- .config
- .gnupg
- .local
- .pki
- .ssh
- .vscode
- cs470
  - des470
  - Lab2
  - Lab3
  - Lab4
  - Lab5
    - Diego_libFC.c
    - Diego_libFC.h
    - Diego_libFC.o
    - diego_testapp
    - Diego_testFC.c
    - Diego_testFC.o
    - DiegoG_Trevino_I...
    - Makefile
  - source470
  - hello.sh
  - Quiz4
  - Quiz4.c
- des470
- Desktop
- Documents
- Downloads

OUTLINE

TIMELINE

Makefile ~/.../Lab3 | sjf.c | Diego_testFC.c | Diego_libFC.h | Diego_libFC.c | Makefile ~/.../Lab5

cs470 > Lab5 > Diego_testFC.c

```c
37  static const char* intro_text(void) {
38      return
43          "It also reinforced the importance of validating each step (like checking open handles)
44          "Overall, it gave me a clearer picture of how real file systems manage data and enforce
45  }
46
47  // prints the menu
48  static void print_menu(int fd_state) {
49      printf("\n============================\n");
50      printf(" libFC Test App (Diego)\n");
51      printf("============================\n");
52      printf("File: %s\n", FILENAME);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

./diego_testapp - Lab5

```
diego@codebind:~/cs470/Lab5$ make
./diego testapp
Press ENTER to continue...3

============================
 libFC Test App (Diego)
============================
File: DiegoG_Trevino_Introduction.txt
Open handle state: OPEN

1) fileCreate  (create new file)
2) fileOpen    (open existing file)
3) fileWrite   (write introduction)
4) fileRead    (read + print file)
5) fileClose   (close file)
6) fileDelete  (delete file)
0) Exit

Enter choice: 3

Writing to file...
Wrote 701 bytes.

Press ENTER to continue...
```

diego master* 0 0 Ln 45, Col 2 Spaces: 4 UTF-8 LF {} C Finish Setup

Ubuntu 24.04.3 LTS

Mar 3 05:38

File Edit Selection View Go Run ... diego

EXPLORER

DIEGO
- .cache
- .config
- .gnupg
- .local
- .pki
- .ssh
- .vscode
- cs470
  - des470
  - Lab2
  - Lab3
  - Lab4
  - Lab5
    - Diego_libFC.c
    - Diego_libFC.h
    - Diego_libFC.o
    - diego_testapp
    - Diego_testFC.c
    - Diego_testFC.o
    - DiegoG_Trevino_I...
    - Makefile
  - source470
  - hello.sh
  - Quiz4
  - Quiz4.c
- des470
- Desktop
- Documents
- Downloads
- Music
- OS-Lab1

OUTLINE

TIMELINE

Makefile ~/.../Lab3 | sjf.c | Diego_testFC.c | Diego_libFC.h | Diego_libFC.c | Makefile ~/.../Lab5 | Makefile ~/.../Lab4 | rr

cs470 > Lab5 > Diego_testFC.c

```c
37  static const char* intro_text(void) {
38      return
43          "It also reinforced the importance of validating each step (like checking open handles) before proceeding to the next
44          "Overall, it gave me a clearer picture of how real file systems manage data and enforce correct usage patterns.\n";
45  }
46
47  // prints the menu
48  static void print_menu(int fd_state) {
49      printf("\n============================\n");
50      printf(" libFC Test App (Diego)\n");
51      printf("============================\n");
52      printf("File: %s\n", FILENAME);
53      printf("Open handle state: %s\n", (fd_state >= 0) ? "OPEN" : "CLOSED");
54      printf("\n1) fileCreate  (create new file)\n");
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

./diego_testapp - Lab5

```
diego@codebind:~/cs470/Lab5$ make
./diego testapp
2) fileOpen    (open existing file)
3) fileWrite   (write introduction)
4) fileRead    (read + print file)
5) fileClose   (close file)
6) fileDelete  (delete file)
0) Exit

Enter choice: 4

Reading file...
---- START ----
Hi, my name is Diego G. Trevino and I am a Computer Science student at Central Washington University. In this lab, I built a user-level file system libr
ary test application that demonstrates basic file operations. This program uses menu-driven prompts to guide the user through creating, opening, writing
, reading, and closing a file.

This project helped me understand how file system calls can be emulated at the user level while still handling errors safely. It also reinforced the imp
ortance of validating each step (like checking open handles) before proceeding to the next operation. Overall, it gave me a clearer picture of how real
file systems manage data and enforce correct usage patterns.

---- END ----

Press ENTER to continue...
```

diego master* 0 0 Ln 45, Col 2 Spaces: 4 UTF-8 LF {} C Finish Setup

Ubuntu 24.04.3 LTS

Mar 3 05:38

cs470 > Lab5 > Diego_testFC.c

```c
static const char* intro_text(void) {
    return
        "It also reinforced the importance of validating each step (like checking open handles) before proceeding to the next
        "Overall, it gave me a clearer picture of how real file systems manage data and enforce correct usage patterns.\n";
}

// prints the menu
static void print_menu(int fd_state) {
    printf("\n============================\n");
    printf(" libFC Test App (Diego)\n");
    printf("============================\n");
    printf("File: %s\n", FILENAME);
    printf("Open handle state: %s\n", (fd_state >= 0) ? "OPEN" : "CLOSED");
    printf("\n1) fileCreate  (create new file)\n");
```

```
diego@codebind:~/cs470/Lab5$ make
./diego testapp
---- END ----

Press ENTER to continue...

============================
 libFC Test App (Diego)
============================
File: DiegoG_Trevino_Introduction.txt
Open handle state: OPEN

1) fileCreate  (create new file)
2) fileOpen    (open existing file)
3) fileWrite   (write introduction)
4) fileRead    (read + print file)
5) fileClose   (close file)
6) fileDelete  (delete file)
0) Exit

Enter choice: 5
File closed.

Press ENTER to continue...
```

Ln 45, Col 2 Spaces: 4 UTF-8 LF {} C Finish Setup

Ubuntu 24.04.3 LTS

Mar 3 05:39

cs470 > Lab5 > Diego_testFC.c

```c
static const char* intro_text(void) {
    return
        "It also reinforced the importance of validating each step (like checking open handles) before proceeding to the next
        "Overall, it gave me a clearer picture of how real file systems manage data and enforce correct usage patterns.\n";
}

// prints the menu
static void print_menu(int fd_state) {
    printf("\n============================\n");
    printf(" libFC Test App (Diego)\n");
    printf("============================\n");
    printf("File: %s\n", FILENAME);
    printf("Open handle state: %s\n", (fd_state >= 0) ? "OPEN" : "CLOSED");
    printf("\n1) fileCreate  (create new file)\n");
```

```
diego@codebind:~/cs470/Lab5$ make
./diego testapp
File closed.

Press ENTER to continue...

============================
 libFC Test App (Diego)
============================
File: DiegoG_Trevino_Introduction.txt
Open handle state: CLOSED

1) fileCreate  (create new file)
2) fileOpen    (open existing file)
3) fileWrite   (write introduction)
4) fileRead    (read + print file)
5) fileClose   (close file)
6) fileDelete  (delete file)
0) Exit

Enter choice: 6
File deleted.

Press ENTER to continue...
```

Ln 45, Col 2 Spaces: 4 UTF-8 LF {} C Finish Setup